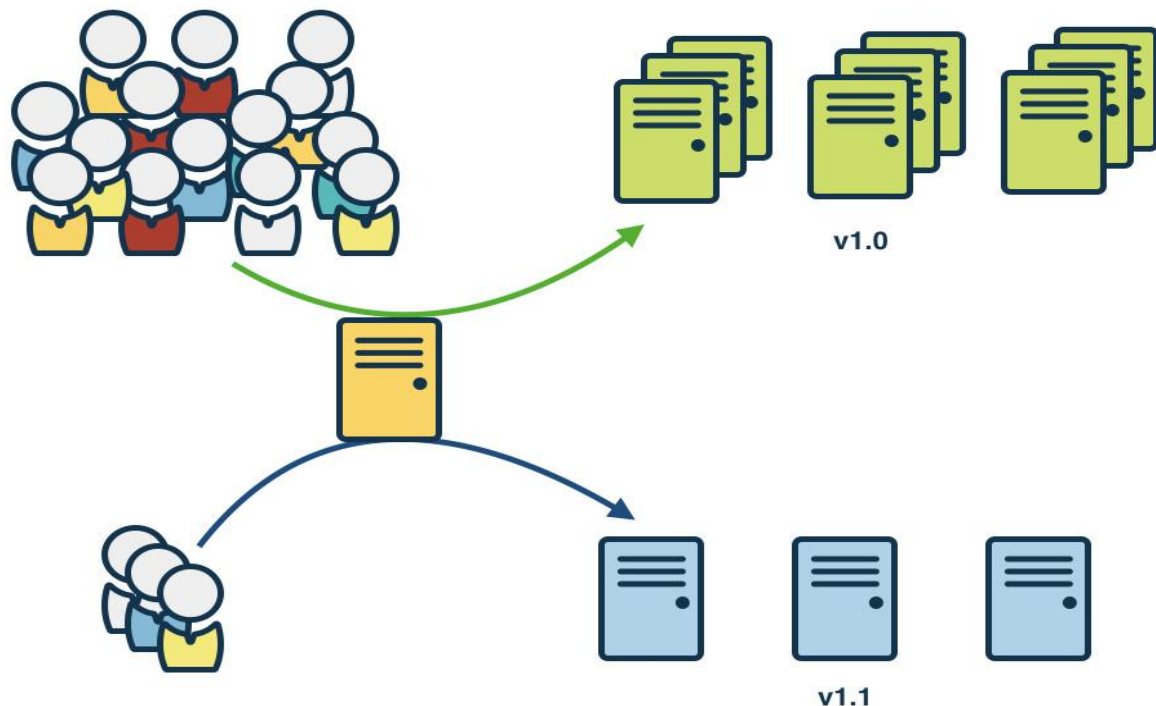




Canary Deployment



Canary Deployment is a strategy used to **release new software updates gradually** to a **small subset of users** before rolling it out to everyone.

Why is it called "Canary"?

The term comes from "**canary in a coal mine.**" Miners used to carry canaries into coal mines as an early warning system. If the bird died, it indicated the presence of toxic gases, alerting miners to evacuate.

Similarly, in software:

- You deploy the update to a small group (the "canary").
- If there are no issues, you gradually **increase the rollout** to more users.
- If problems arise, you can **roll back** without impacting all users.

When to Use Canary Deployment?

- When releasing a **new version of a web app**, API, or microservice.
- When you want to **test in production** with real users.



- When **minimizing risk** of system-wide failure is critical.
- When you want **automated rollback** in case of errors.

Key Concepts in Canary Deployment:

Concept	Description
Old Version (Stable)	The currently running version used by all users.
Canary Version (New)	The new version being tested with a small group.
Traffic Splitting	Dividing incoming user traffic between old and new versions.
Monitoring	Checking logs, metrics, errors, and user feedback from canary users.
Rollback	Automatically or manually switching back to the stable version if issues are detected.

Detailed Step-by-Step Description

1. Build & Test Phase

- Developers push code to the repository.
- CI pipeline triggers build.
- Run unit, integration, and static analysis tests.

2. Image Creation & Push

- Docker image for the new version (v2.0) is built.
- It is pushed to a container registry.

3. Stable Deployment

- Version v1.0 is already running in production and serving 100% traffic.

4. Canary Deployment



- Deploy version v2.0 alongside version v1.0.
- Route a small percentage of real user traffic (e.g., 5%–10%) to v2.0.

5. Observability

- Monitor:
 - Error rates
 - Latency
 - CPU/Memory usage
 - User behavior
 - Log anomalies

6. Decision Point

-  If everything is healthy:
 - Gradually increase traffic to the canary:
 - 25% → 50% → 75% → 100%
-  If issues are found:
 - Immediately stop canary version v2.0
 - Route 100% traffic back to stable v1.0

7. Finalization

- If the rollout is successful:
 - Decommission the old version v1.0
 - v2.0 becomes the new stable version

Practical Example Using Kubernetes (Real-World Scenario)

Scenario:

You have a Kubernetes deployment for a `httpd` . You want to roll out **nginx** canary while most users still get `httpd`.



✓ Goal of This Practical

1. Create 2 deployments:
 - One with httpd (canary v1)
 - One with nginx (canary v2)
2. Use kubectl dry-run to generate YAMLs and edit them:
 - Add the **same label** to both deployments (e.g., app: web)
3. Create a Service with a selector using the common label app: web
4. Result: Service will load-balance traffic between httpd and nginx pods (50-50 or however replicas are defined)

⚙ Step-by-Step Practical Setup

◇ Step 1: Create HTTPD Deployment YAML (v1)

- `kubectl create deployment httpd-deployment --image=docker.io/httpd --dry-run=client -o yaml > httpd.yaml`

```
controlplane:~$ ls
filesystem httpd.yaml
```

◆ Step 2: Edit httpd.yaml

Change the YAML to add a label **team:Unnati** :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: httpd-deployment
  name: httpd-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpd-deployment
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: httpd-deployment
        team: unnati
    spec:
      containers:
        - image: docker.io/httpd
          name: httpd
          resources: {}
status: {}
```



◇ Step 3: Apply Deployment

- `kubectl apply -f httpd.yaml`

```
controlplane:~$ kubectl apply -f httpd.yaml
deployment.apps/httpd-deployment created
```

- `Kubectl get pods`

```
controlplane:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
httpd-deployment-ddbbbf56-2mhgn    1/1     Running   0           61s
```

◆ Step 4: Create Service YAML Using Dry-Run

- `kubectl expose deployment httpd-deployment --port=80 --target-port=80 --type=NodePort --name=web-service --dry-run=client -o yaml > service.yaml`

```
controlplane:~$ kubectl expose deployment httpd-deployment --port=80 --target-port=80 --type=NodePort --name=web-service --dry-run=client -o yaml > service.yaml
```

◇ Step 5: Edit service.yaml

Make sure the to add team: Unnati selector:

```
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: httpd-deployment
  name: web-service
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
  selector:
    app: httpd-deployment
    team: unnati
  type: NodePort
status:
  loadBalancer: {}
```



◇ Step 6: Apply the Service

- `kubectl apply -f service.yaml`

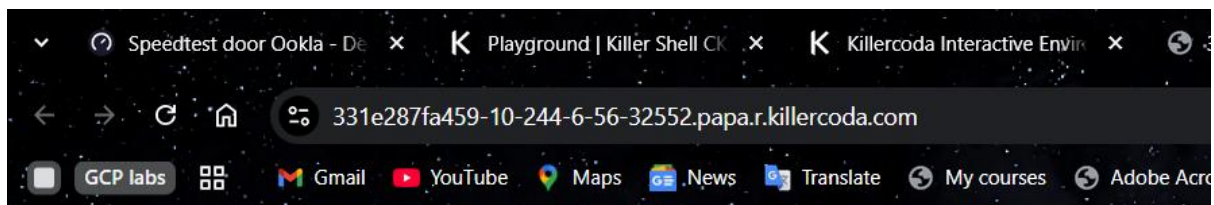
```
controlplane:~$ kubectl apply -f service.yaml
service/web-service configured
```

```
controlplane:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)        AGE
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP         18d
web-service          NodePort    10.110.7.36   <none>         80:32552/TCP    8m37s
```

```
controlplane:~$ kubectl describe svc web-service
Name:                web-service
Namespace:           default
Labels:              app=httpd-deployment
Annotations:          <none>
Selector:            app=httpd-deployment,team=unnati
Type:                NodePort
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.110.7.36
IPs:                 10.110.7.36
Port:                <unset> 80/TCP
TargetPort:          80/TCP
NodePort:            <unset> 32552/TCP
Endpoints:           192.168.1.5:80
Session Affinity:    None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events:              <none>
```

Here we can see the only one endpoint is there which map the httpd-deployment

◇ Step 7: Access the service outside using NodePort



It works!



Scenario

- *Now this is fine, but I updated my application with version 2, where I changed my application server from httpd to Nginx, so how my service works or shifts traffic will happen as follows*

◇ Step 1: Create NGINX Deployment YAML (v2)

- `kubectl create deployment nginx-deployment --image=docker.io/nginx --replicas=3 --dry-run=client -o yaml > nginx.yaml`

```
controlplane:~$ kubectl create deployment nginx-deployment --image=docker.io/nginx --replicas=3 --dry-run=client -o yaml > nginx.yaml
controlplane:~$
controlplane:~$ ls
filesystem  httpd.yaml  nginx.yaml  service.yaml
```

◇ Step 2: Edit nginx.yaml

Make sure it uses the **same label** (team: unnati):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: nginx-deployment
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-deployment
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: nginx-deployment
        team: unnati
    spec:
      containers:
      - image: docker.io/nginx
        name: nginx
        resources: {}
  status: {}
```



◇ Step 3: Apply Deployment

- `kubectl apply -f nginx.yaml`

```
controlplane:~$ kubectl apply -f nginx.yaml
deployment.apps/nginx-deployment created
```

- Verify the deployments & see the pods
 - `Kubectl get pods`

```
controlplane:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
httpd-deployment-ddbbbfb56-9w55w	1/1	Running	0	23m
nginx-deployment-6fd95dc4bd-4tql7	1/1	Running	0	14s
nginx-deployment-6fd95dc4bd-zqmfw	1/1	Running	0	14s
nginx-deployment-6fd95dc4bd-zwcv8	1/1	Running	0	14s

- Check the deployments
 - `Kubectl get deployments`

```
controlplane:~$ kubectl get deployments.apps
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
httpd-deployment	1/1	1	1	24m
nginx-deployment	3/3	3	3	74s

- Edit the service yaml file, remove the (app=https-deployment), and stay with the common selector which we mentioned in both deployments
 - `Vim service.yaml`

```
selector:
team: unnati
```

Make like above

- Now, apply the changes made in the service file
 - `Kubectl apply -f service.yaml`

```
controlplane:~$ kubectl apply -f service.yaml
service/web-service configured
```

```
controlplane:~$ kubectl get svc
```

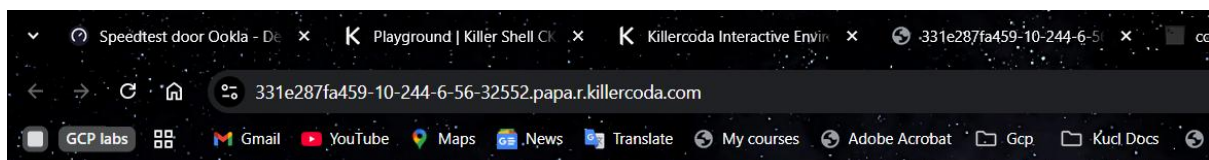
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	18d
web-service	NodePort	10.110.7.36	<none>	80:32552/TCP	8m37s



- Here we can see the detailed info about the service, and here we can see our service is successfully attached to the new version and new deployment called nginx-deployment
 - **Kubectl describe svc web-service**

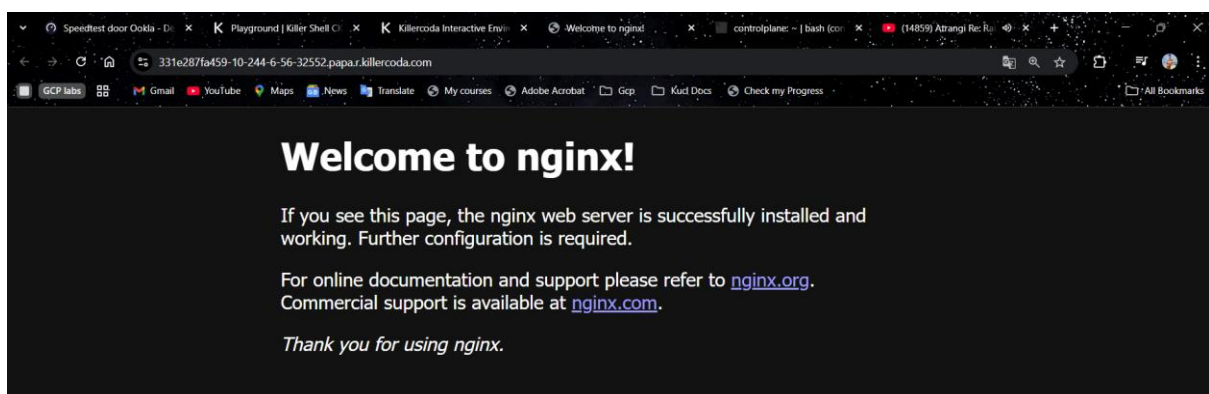
```
controlplane:~$ kubectl describe svc web-service
Name:                web-service
Namespace:           default
Labels:              app=httpd-deployment
Annotations:         <none>
Selector:            team=unnati
Type:               NodePort
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                 10.110.7.36
IPs:                10.110.7.36
Port:               <unset> 80/TCP
TargetPort:         80/TCP
NodePort:           <unset> 32552/TCP
Endpoints:          192.168.1.5:80,192.168.1.6:80,192.168.1.7:80 + 1 more...
Session Affinity:    None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events:             <none>
```

- Now we can verify that both deployments are running on the same service by accessing the service NodePort
 - <http://<NodeIp>:<nodeport>>



It works!

- After refreshing page





- Here we can see the labels of deployment
 - **Kubectl get pods --show-labels**

```
controlplane:~$ kubectl get pods --show-labels
NAME                                READY   STATUS    RESTARTS   AGE   LABELS
httpd-deployment-ddbbbf56-9w55w     1/1     Running   0           28m   app=httpd-deployment,pod-template-hash=ddbbbf56,team=unnati
nginx-deployment-6fd95dc4bd-4tql7   1/1     Running   0           5m29s app=nginx-deployment,pod-template-hash=6fd95dc4bd,team=unnati
nginx-deployment-6fd95dc4bd-zqmfw   1/1     Running   0           5m29s app=nginx-deployment,pod-template-hash=6fd95dc4bd,team=unnati
nginx-deployment-6fd95dc4bd-zwcv8   1/1     Running   0           5m29s app=nginx-deployment,pod-template-hash=6fd95dc4bd,team=unnati
```

Scenario

Now my company decides we have to go with version 2, which means we have to switch all traffic to nginx, so for that we do as follows

- We have to delete older version (httpd-deployment)
 - **Kubectl delete deployment httpd-deployment**

```
controlplane:~$ kubectl delete deployments.apps httpd-deployment
deployment.apps "httpd-deployment" deleted
```

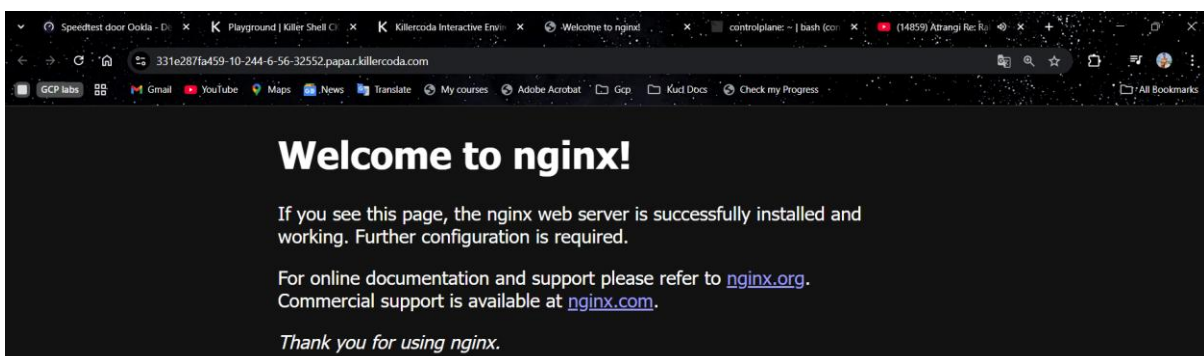
- Verify deleted deployment

```
controlplane:~$ kubectl get deployments.apps
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment                   3/3      3             3           6m51s
```

- Check the pods

```
controlplane:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6fd95dc4bd-4tql7   1/1     Running   0           7m2s
nginx-deployment-6fd95dc4bd-zqmfw   1/1     Running   0           7m2s
nginx-deployment-6fd95dc4bd-zwcv8   1/1     Running   0           7m2s
```

- Now, if we access the same service and refresh many times, but we get the same application means the stable version of the application





How This Is a Canary Deployment

- You're running **two versions** of the app: httpd and nginx.
- They both **share the same label** (team: unnati), so traffic is split.
- You can **adjust traffic** by changing replica count:
 - 1 pod for httpd, 3 pods for nginx → 75% traffic to nginx.
- You can **scale down** one to fully shift traffic.
 - `kubectl scale deployment httpd-deploy --replicas=0`

Summary of Key Concepts

Component	Value
Canary v1	httpd-deploy
Canary v2	nginx-deploy
Common Label	team: unnati
Service	web-service
Routing Logic	Based on label selector
Control	Done via number of replicas

